Phân loại đa nhãn họ mã độc bằng mô hình học sâu kết hợp cơ chế Attention

Nov 12, 2026 · @Lương

Môn học	Chuyên đề nghiên cứu về Phân loại mã độc (IT3003.11.CH)
Đề tài	TAKA10: Phân loại đa nhãn họ mã độc bằng mô hình học sâu kết hợp cơ chế Attention
Học viên	Lê Tuấn Lương
Mã số học viên	250202013
Mã nguồn	luonglt20/MAL_Class

Mục lục

1. Tổng quan
2. Cấu trúc repository
3. Dữ liệu
4. Kiến trúc mô hình
5. Quy trình huấn luyện, hiệu chỉnh và đánh giá
6. Ứng dụng và suy luận
7. Kết quả thực nghiệm: 7.1–7.4 kết quả trên dữ liệu v1 · 7.5 dữ liệu v2 và baseline TF-IDF · 7.6 test độc lập · 7.7 phân tích lỗi · 7.8 các phiên bản và kết quả cuối
8. Hạn chế và hướng phát triển: 8.1 hạn chế · 8.2 hướng phát triển · 8.3 hướng hành vi · 8.4 dữ liệu nhãn tin cậy hơn
9. Kết luận
10. Tài liệu tham khảo

1. Tổng quan

MAL_Class phân loại họ (family) mã độc Windows PE bằng một mô hình Hybrid Attention ba nhánh, kết hợp bằng chứng chữ ký, cấu trúc PE tĩnh và hành vi động từ sandbox. Đến ngày 13/11/2026, mô hình đạt top-1 validation 64–70% trên 181 mẫu (dữ liệu v1) và 64,2% trên 670 mẫu (dữ liệu v2, 6 epoch); baseline TF-IDF + logistic regression đạt 71,6% trên cùng 670 mẫu, và 73,2% / 70,2% trên hai tập test độc lập chấm một lần. Mục tiêu nghiệm thu là trên 85% với đánh giá độc lập.

Bài toán. Đầu vào là một file PE hoặc report sandbox CAPE. Đầu ra là danh sách họ mã độc xếp hạng (đa nhãn), hành vi suy luận được, và quyết định có trả lời hay từ chối (abstention). Hệ thống không bao giờ thực thi mẫu; SHA-256 chỉ dùng để định danh và khử trùng lặp.

Mục tiêu nghiệm thu (đặt ra cho đề tài):

- Top-1 trên 85% với tập test độc lập, không dùng test để chọn mô hình.
- Báo cáo tách riêng kết quả PE-only và hybrid, kèm phạm vi nhãn, mẫu số, độ phủ khi từ chối và nguồn gốc split.
- Giao diện local nhận file PE hoặc dataset, trích xuất đặc trưng an toàn và phân loại bằng mô hình attention.

Đóng góp chính của đồ án:

1. Kiến trúc ba nhánh (Signature, Static PE, Dynamic/Behavior) với self-attention riêng và cross-attention hai chiều giữa các nhánh, thay vì cộng điểm từng nhánh.
2. Pipeline dữ liệu WinMET đọc report thẳng từ archive 7z có giới hạn tài nguyên, chuẩn hóa có phiên bản (hiện là v4) và chia tập theo archive group để tránh rò rỉ.
3. Quy trình đánh giá chặt: split cố định bằng fingerprint, hiệu chỉnh ngưỡng trên calibration, conformal prediction, test chỉ chấm một lần có lock file.
4. Dashboard Streamlit phân tích PE tĩnh trong sandbox, tự chọn mô hình theo loại đầu vào và từ chối khi đầu vào không tương thích.

Dự án MAL_dectect trước đó của học viên chỉ được dùng làm nguồn tham khảo ý tưởng; toàn bộ kết quả trong báo cáo này thuộc MAL_Class.

2. Cấu trúc repository

Mã nguồn lõi nằm trong package Python `src/malware_hybrid` (35 module, khoảng 11.400 dòng), đi kèm 27 script thí nghiệm, 49 file test và một dashboard Streamlit. Repo: luonglt20/MAL_Class.

Thư mục	Nội dung
`src/malware_hybrid/`	Package lõi: parse, chuẩn hóa, tensor hóa, mô hình, train, hiệu chỉnh, suy luận
`scripts/`	Thí nghiệm WinMET, manifest mở rộng, ablation, A/B, chấm test một lần
`Source_Code/`	Dashboard Streamlit `hybrid_attention_dashboard.py` và các bản tham khảo cũ

Thư mục	Nội dung
`tests/`	Unit/integration test (pytest), fixture report CAPE
`configs/`	Khóa checksum dataset, registry mô hình suy luận, cấu hình mặc định
`docs/`	Workflow WinMET, trạng thái triển khai, ghi chép validation, handoff
`artifacts/`, `models/`, `Data/`	Kết quả, checkpoint và dữ liệu thô (bị `.gitignore`, không đưa lên GitHub)

Các module chính, nhóm theo tầng xử lý:

Tầng	Module	Vai trò
Thu thập dữ liệu	`winmet.py`, `dataset_sources.py`, `ember_adapter.py`, `serialization.py`	Tải có khóa checksum, đọc report từ archive 7z có giới hạn, chuyển EMBER2024 về schema chung, ghi JSONL
Chuẩn hóa	`normalization.py`, `handles.py`, `schema.py`, `signature_policy.py`	Parse CAPE, chuẩn hóa tham số bảo vệ quyền riêng, giải handle process, lọc rule YARA an toàn
Đặc trưng	`graph.py`, `evidence.py`, `fuzzy.py`, `pe_static.py`, `static_dialect.py`	Dựng đồ thị provenance theo thời gian, kiểm chuỗi băng chứng, prototype TLSH/ssdeep, trích PE tĩnh
Mô hình	`dataset.py`, `model.py`, `baselines.py`	Vocabulary và tensor hóa, mô hình Hybrid Attention, 7 baseline so sánh
Huấn luyện, đánh giá	`training.py`, `calibration.py`, `metrics.py`, `evaluation.py`, `faithfulness.py`, `readout.py`, `prototype_readout.py`	Train tái lập, conformal và ngưỡng, metric đa nhãn, ablation, kiểm định attention, readout top-1 cố định
Suy luận, ứng dụng	`pipeline.py`, `inference_service.py`, `batch_analysis.py`, `jsonl_analysis.py`, `isolated_pe.py`, `cli.py`	Pipeline end-to-end, định tuyến mô hình, phân tích batch, parser PE trong sandbox, CLI `malware-hybrid`

Tầng	Module	Vai trò
Ánh xạ khung	`mappings.py`, `framework_sync.py`, `deployment_audit.py`	Crosswalk ATT&CK → MBC → NIST SP 800-53, kiểm tra quyết định hiển thị cho người dùng

Môi trường: Python 3.11, PyTorch 2.15 trên Apple Silicon (MPS), quản lý bằng `uv`; test chạy bằng `pytest` (340 test).

3. Dữ liệu

Nguồn chính là WinMET v3: 31.844 report sandbox CAPE trong 5 volume [1], [2], sinh bởi sandbox CAPEv2 [4], nhãn họ lấy từ AVClass consensus [3]. Sau khi mở rộng ngày 12/11/2026, tập train tăng từ 1.676 lên 8.500 report.

Dataset	Vai trò	Ghi chú
WinMET v3 (Zenodo, GPL-3.0-or-later)	Train và đánh giá mô hình hybrid	Report CAPE có `static.pe`, `target.file.pe` và chuỗi event; dùng volume 1 và 2
EMBER2024 Win32 [5], [6]	Mô hình chỉ có đặc trưng tĩnh	Đã khóa checksum, lấy mẫu reservoir theo tuần; TTP yếu không dùng làm nhãn âm
MalBehavD-V1	Thí nghiệm phụ trên chuỗi API	Không ghép với WinMET; không dùng cho phân loại họ

Đọc dữ liệu an toàn. Mỗi volume WinMET nén khoảng 2,4 GB nhưng giải nén ra khoảng 154 GB. Lệnh `prepare-winmet` kiểm tra MD5 và SHA-256 đã khóa, chỉ đọc các member JSON tên SHA-256, giải nén vào bộ nhớ có giới hạn rồi ghi ngay ra JSONL đã chuẩn hóa. Không có file thực thi nào được giải nén ra đĩa.

Chuẩn hóa có phiên bản. Mỗi checkpoint ghi lại phiên bản chuẩn hóa đã dùng khi train, và guard từ chối report khác phiên bản.

- `cape-pe-locations-v2`: đọc cả hai layout PE, phục hồi imphash vào metadata. Trên cùng 2.500 report, số token nguyên tử tăng từ 17.521 lên 367.560.
- `cape-process-relations-v3`: nhận diện pseudo-handle của chính process, không coi thao tác tự ghi bộ nhớ là injection.
- `cape-resource-relations-v4` (đang dùng): nối thao tác mở file với các thao tác sau đó qua handle cục bộ của process.

- Hash, verdict của vendor và đường dẫn cá nhân không bao giờ trở thành token của mô hình.

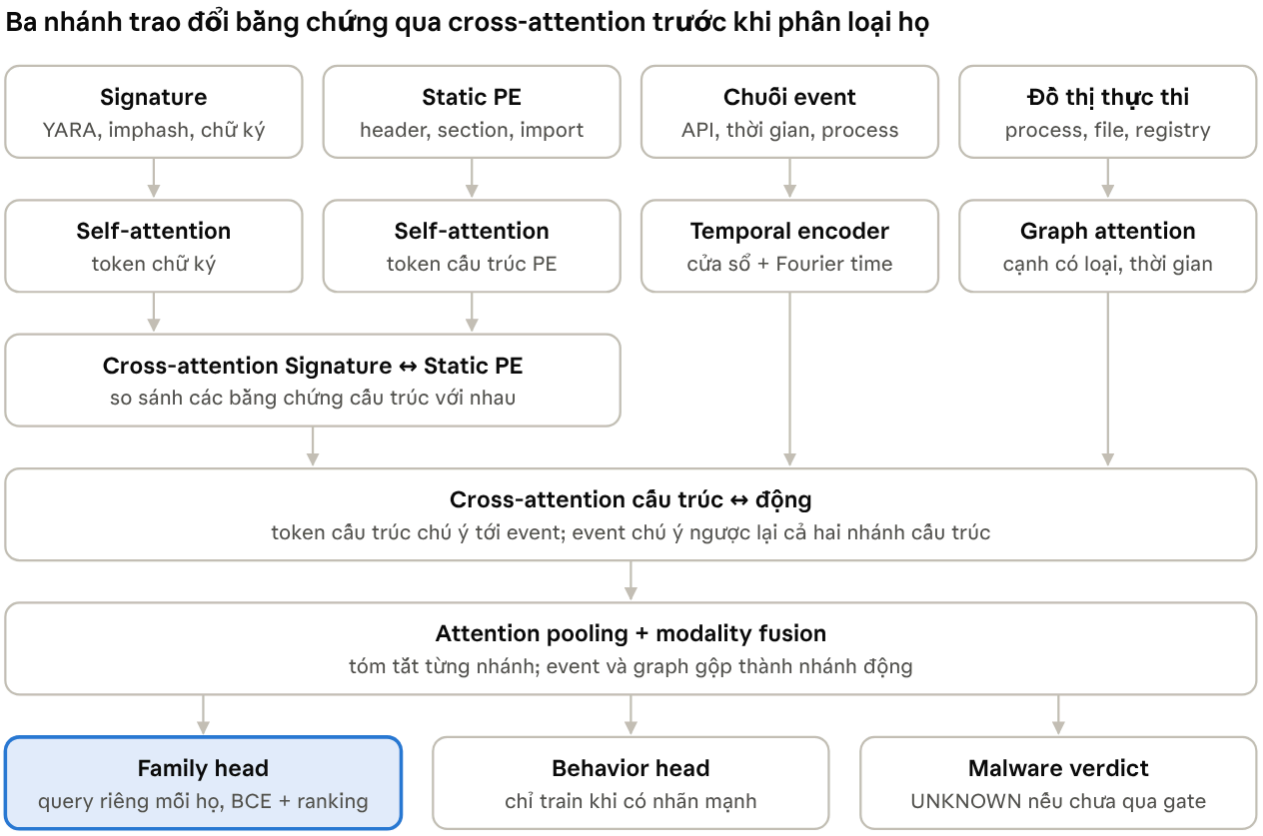
Chia tập theo archive group. WinMET nén theo solid block, và các report trong cùng block thường tương quan. Vì vậy cả block được gán vào một partition, và manifest lưu fingerprint SHA-256 của danh sách mẫu mỗi partition. Loader kiểm tra lại fingerprint mỗi lần đọc và không dựng object cho các partition test.

Partition	Manifest v1 (29/10)	Manifest v2 (12/11)
Train	1.676 report / 26 block	8.500 report / 126 block
Validation	246 / 3 block	783 / 11 block
Calibration	216 / 3 block	216 / 3 block (giữ nguyên)
Test v1 (`withheld_internal_test`)	362 / 5 block	362 / 5 block (giữ nguyên)
Test v2 (`withheld_test_v2`)	–	779 / 12 block, lấy từ volume 1

Manifest v2 giữ nguyên mọi gán block của v1, khử trùng theo SHA-256 (0 dòng bị loại) và có 153 họ với ít nhất 5 mẫu train. Khoảng 58–63% mẫu train mới có imphash trùng với một mẫu trong test; điều này thường do packer hoặc thư viện dùng chung, nhưng cần nêu khi báo cáo test.

4. Kiến trúc mô hình

Mô hình `HybridAttentionModel` mã hóa riêng ba nguồn bằng chứng, cho chúng so sánh với nhau bằng cross-attention [7] hai chiều, rồi mới đưa ra quyết định. Nhờ vậy mô hình có thể nối một import đáng ngờ với sự kiện tải, ghi rồi thực thi file, thay vì chỉ cộng ba điểm tổng.



Kiến trúc HybridAttentionModel ba nhánh · src/malware_hybrid/model.py

Các thành phần chính:

- **Nhánh Signature và Static PE** (`StaticEvidenceEncoder`): mỗi token là một bằng chứng nguyên tử (tên rule YARA cấu trúc, đặc tính section, DLL/import, entropy). Budget 256 token và giữ đủ các loại đặc trưng, nên danh sách import dài không chiếm hết chỗ.
- **Nhánh thời gian** (`TemporalHierarchicalEncoder`): tối đa 2.048 event, chia cửa sổ 256 có chồng lấp. Mỗi event mang API, thời điểm (mã hóa Fourier), trạng thái thành công, số lần lặp, process/thread và resource; `RelationalAttentionBlock` thêm bias khi hai event cùng thao tác một tài nguyên.
- **Nhánh đồ thị** (`TemporalGraphEncoder` , `EdgeGraphAttentionLayer`): node là process, file, registry, network; cạnh có loại, thời gian, trạng thái và cờ nhân quả; lớp attention trên đồ thị mở rộng Graph Attention Network [8] với đặc trưng cạnh.
- **Cross-attention**: Signature ↔ Static PE trước, sau đó hai nhánh cấu trúc ↔ event/graph. Ablation ở mục 7 cho thấy tắt bước này làm macro-F1 giảm từ 24,1% xuống 14,4%.
- **Modality dropout**: khi train, ngẫu nhiên che cả một nhánh trước cross-attention (luôn giữ ít nhất một nhánh), để mô hình không phụ thuộc vào một nguồn duy nhất.

- **Ba đầu ra**: family head dạng `labelwise` (mỗi họ một query học được), behavior head và malware head. Behavior head chỉ được train khi tập train có cả nhãn dương và âm mạnh.

Hàm mất mát. Loss họ là BCE đa nhãn có mask (bỏ qua nhãn chưa biết), cộng thêm một thành phần ranking cross-entropy để đẩy họ đúng lên đầu:

$$\mathcal{L}_{family} = \mathcal{L}_{BCE} + \lambda \, \mathcal{L}_{rank}, \quad \lambda = 0.5$$

Cấu hình đang dùng cho WinMET: `d_model` = 32, 4 head, 2 lớp mỗi encoder, `family_head_variant` = `labelwise`, `modality_dropout` = 0, lựa chọn checkpoint theo family mAP trên validation.

Baseline so sánh (`baselines.py`): indicator rule, bag-of-tokens, BiLSTM, temporal transformer, graph attention, sequence + graph và full hybrid attention, cùng vocabulary, loss và budget epoch.

5. Quy trình huấn luyện, hiệu chỉnh và đánh giá

Quy trình thực nghiệm tách bốn vai trò dữ liệu: train để học tham số, validation để chọn mô hình, calibration để hiệu chỉnh ngưỡng, và test chỉ dùng một lần cho kết quả cuối cùng. Mọi thí nghiệm cố định seed (mặc định 44) và lưu cấu hình, checksum dữ liệu cùng lịch sử từng epoch.

5.1. Các bước huấn luyện

1. **Nạp dữ liệu theo manifest.** Kiểm tra SHA-256 của file nguồn và fingerprint từng partition. Các partition test không được dựng thành object.
2. **Tensor hóa** (`ReportTensorizer`). Vocabulary token, cạnh và loại node chỉ được fit trên tập train. Họ cần ít nhất 5 mẫu train mới vào vocabulary; họ hiếm hơn được xem là ngoài vocabulary (OOV).
3. **Tối ưu.** AdamW, learning rate 3×10^{-4}, weight decay 10^{-4}, gradient clipping 1,0, batch 4. Trọng số dương của từng nhãn được tính từ tần suất trên train để bù mất cân bằng lớp.
4. **Chọn checkpoint.** Sau mỗi epoch, mô hình được đánh giá trên validation; checkpoint có family mAP cao nhất được giữ. Có tùy chọn dừng sớm theo số epoch không cải thiện.
5. **Hiệu chỉnh trên calibration.** Fit ngưỡng từng họ với giới hạn tỷ lệ báo động giả (FPR), ngưỡng malware, và tập dự đoán conformal.
6. **Chấm test một lần.** Script `evaluate_winmet_expansion_test.py` tạo lock file theo checkpoint và partition, nên không thể chấm lại để chọn mô hình.

5.2. Định lượng bất định và chính sách kết luận

- **Simultaneous split-conformal** cho vector nhãn quan sát một phần [9]: tránh trường hợp hàng trăm tập conformal biên 90% cùng lúc được gọi là "confirmed".
- **Khoảng tin cậy Wilson** [10] cho các tỷ lệ đo trên mẫu nhỏ.
- **Chính sách thận trọng:** verdict `MALICIOUS` / `BENIGN` trả `UNKNOWN` cho đến khi checkpoint vượt release gate độc lập. Behavior chỉ được `confirmed` khi có điểm từ behavior head đã train và chuỗi bằng chứng đầy đủ.

5.3. Metric

Metric chính là top-1 và top-3 accuracy trên các mẫu có họ nằm trong vocabulary, macro-F1 và mean Average Precision (mAP) cho bài toán đa nhãn. Mẫu số luôn được báo cáo kèm. Selective accuracy (chỉ tính trên mẫu được chấp nhận) luôn đi kèm độ phủ (coverage) và không được thay cho top-1 toàn phần. Baseline tham chiếu là bộ phân loại theo tần suất họ trên train (training prior), đạt top-1 22,1%.

5.4. Tối ưu tài nguyên (bổ sung ngày 11/11/2026)

Trên máy 16 GB RAM, tiến trình train cũ chiếm khoảng 34 GB và phải swap liên tục. Đo trên 100 report cho thấy mỗi report thô chiếm 2,66 MB, trong khi bản đã tensor hóa chỉ 0,34 MB. Hai tùy chọn được thêm vào:

- `pre_encode` kèm giải phóng event thô sau khi tensor hóa.
- `mps_empty_cache_every`: trả bộ nhớ đệm của MPS định kỳ.

Cả hai được kiểm chứng là không đổi kết quả số học: trên cùng môi trường, train loss ở batch 41 trùng nhau đến 7 chữ số (2,5674453). Thời gian mỗi epoch giảm từ 13–15 phút xuống 6–7 phút.

6. Ứng dụng và suy luận

Hệ thống được triển khai thành một dashboard Streamlit chạy cục bộ (`Source_Code/hybrid_attention_dashboard.py`) và một CLI `malware-hybrid`. Nguyên tắc thiết kế là không bao giờ thực thi mẫu, và từ chối trả lời thay vì đoán khi đầu vào vượt ngoài phạm vi đã kiểm chứng.

6.1. Chức năng

- **Phân tích mẫu PE:** nhận một hoặc nhiều file (tối đa 50 MiB/file), phân tích tĩnh và hiển thị họ dự đoán, bằng chứng và trọng số attention.

- **Phân tích batch:** upload hoặc thư mục cục bộ (tối đa 500 file, 256 MiB, không theo symlink), xuất kết quả và lỗi ra CSV. Nội dung có dạng công thức bị escape để chống CSV injection.
- **Phân loại dataset JSONL** (CAPE hoặc đã chuẩn hóa): giới hạn 8 MiB/dòng, 256 MiB/lượt, 500 mẫu.
- **Ánh xạ khung tham chiếu:** hành vi được nối sang MITRE ATT&CK 19.2 [12], MBC và NIST SP 800-53 Rev. 5.1.1 [13], dùng bản tải về đã khóa SHA-256.

6.2. Định tuyến mô hình theo đầu vào

Dashboard chọn mô hình dựa trên bằng chứng đã parse, không dựa trên tên file. Registry `configs/inference_models.json` khóa từng checkpoint bằng đường dẫn, kích thước và SHA-256.

Đầu vào sau khi parse	Mô hình được dùng
File PE, không có trace thực thi	Signature–Static Attention, train trên EMBER2024
JSON CAPE/WinMET có chuỗi event	Hybrid Attention ba nhánh, train trên WinMET
JSON chỉ có bằng chứng tĩnh	Mô hình tĩnh; không tự tạo event giả
Chưa cài checkpoint phù hợp	Chỉ xuất bằng chứng, họ `UNKNOWN`, kèm lý do

6.3. Cơ chế an toàn và từ chối

- **Parser PE trong sandbox:** trên macOS, việc parse PE chạy trong tiến trình con bị `sandbox-exec` cấm mạng, cấm ghi file, cấm tạo tiến trình và cấm đọc thư mục home. Lỗi hoặc quá thời gian đều dẫn tới từ chối (fail closed).
- **Guard vocabulary:** nếu dưới 50% token đầu vào được checkpoint nhận diện, mô hình không chạy và kết quả là `UNKNOWN`. Đây là kiểm tra tương thích ban đầu, chưa phải bộ phát hiện ngoài phân phối (OOD) đã kiểm định.
- **Guard phiên bản chuẩn hóa:** checkpoint từ chối trace có phiên bản chuẩn hóa khác với lúc train.
- **Static dialect:** parser PE cục bộ sinh token kiểu EMBER, còn checkpoint WinMET học token kiểu CAPE `static.pe`. Module `static_dialect.py` dựng lại khối CAPE từ cấu trúc đã parse, nâng độ phủ token từ 44% lên 89%.
- **Từ chối đầu vào chỉ có static cho checkpoint động:** trên validation, khi bỏ toàn bộ event, head của mô hình WinMET chỉ đạt 21,5% top-1, thấp hơn baseline prior 22,1%. Vì vậy pipeline trả `UNKNOWN` thay vì đưa ra dự đoán không đáng tin.

6.4. Readout top-1 cố định

Quy tắc chọn họ top-1 (head của mô hình hoặc kNN trên embedding, cùng ngưỡng chấp nhận) được lưu thành file sidecar khóa SHA-256 với checkpoint. Prototype kNN chỉ lấy từ train và ngưỡng được fit trên calibration. Nhờ đó, giao diện hiển thị đúng quy tắc sẽ dùng để chấm test.

6.5. Bản demo

Bản demo dùng checkpoint run3 (epoch 10, chuẩn hóa `cape-resource-relations-v4`) đã đăng ký trong registry, cùng mô hình tĩnh EMBER2024 cho file PE. Trên máy Apple Silicon, mỗi report được phân tích trong khoảng 0,1 giây.

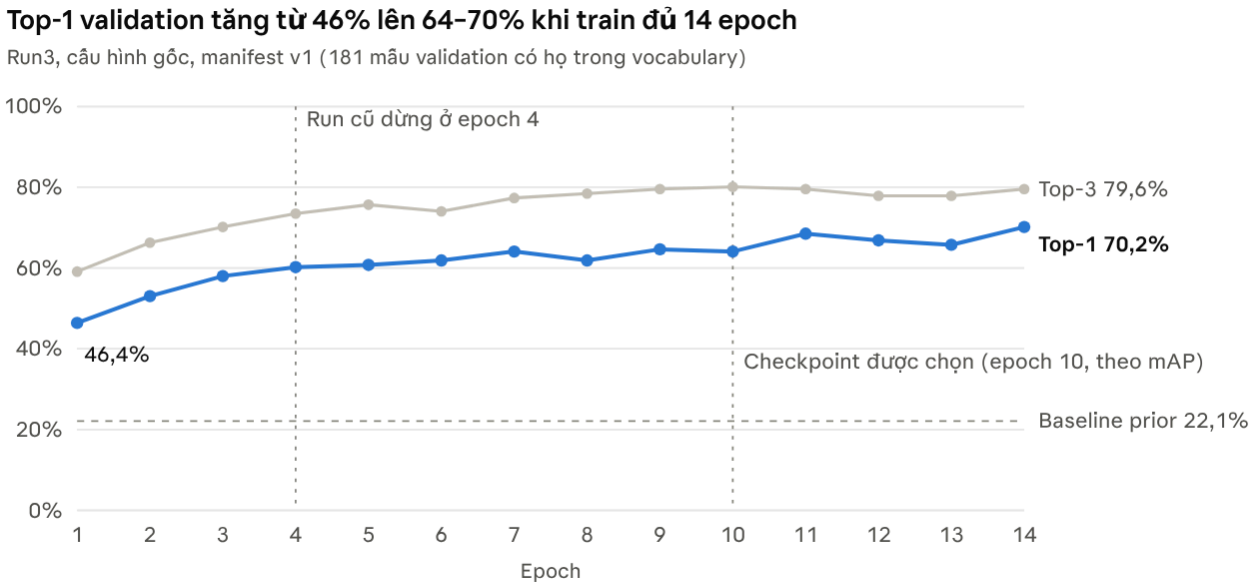
1. Chạy dashboard: `PYTHONPATH=src .venv/bin/python -m streamlit run Source_Code/hybrid_attention_dashboard.py`.
2. Mở mục **Phân loại dataset JSONL**, nhập `artifacts/demo/winmet_validation_demo_20.jsonl`, chọn **Prepared JSONL của MAL_Class**, bấm **Phân loại JSONL**. File gồm 20 mẫu lấy từ validation, không phải từ tập test.
3. Hoặc upload một file PE để xem luồng phân tích tĩnh trong sandbox; dashboard tự chọn mô hình tĩnh.

Checkpoint demo dùng readout top-1 cố định theo head mô hình, ngưỡng 0,8975 fit trên calibration. Trên toàn bộ 181 mẫu validation, readout này đạt top-1 64,1%; khi chỉ trả lời mẫu vượt ngưỡng, đúng 90,8% với độ phủ 54,1%, và chỉ nhận nhầm 3/47 mẫu thuộc họ ngoài vocabulary. Trên 20 mẫu demo: top-1 đúng 11/20, 9 mẫu được chấp nhận và 8 trong số đó đúng. Con số 20 mẫu chỉ dùng để kiểm tra luồng chạy.

7. Kết quả thực nghiệm

Với cấu hình gốc trên 1.676 mẫu train, mô hình đạt top-1 validation 64–70% và top-3 khoảng 80%, gấp khoảng ba lần baseline prior (22,1%). Trên dữ liệu mở rộng v2, baseline TF-IDF + logistic regression đạt 71,6% và vượt mô hình attention 6 epoch (64,2%, mục 7.5). Trên test độc lập chấm một lần (mục 7.6), TF-IDF + LR đạt 73,2% (test v1) và 70,2% (test v2). Mục tiêu 85% chưa đạt. Trừ mục 7.6, số liệu dưới đây là trên validation với nhãn yếu (AVClass).

7.1. Huấn luyện đến hội tụ (run3)



artifacts/winmet/expanded_training_run3_memopt_20261111.log · epoch 1–13, validation manifest v1

Run cũ dừng ở epoch 4 do tắt máy, khi đường cong còn đang đi lên. Train tiếp đến epoch 14 nâng top-1 lên 64–70%, mAP từ 0,335 lên 0,479 và macro-F1 từ 0,244 lên 0,325–0,405. Checkpoint được chọn theo mAP là epoch 10 (top-1 64,1%, top-3 80,1%); epoch 14 đạt top-1 cao nhất (70,2%) nhưng mAP thấp hơn. Giữa các epoch, top-1 dao động 2–4 điểm; với 181 mẫu, mỗi mẫu tương đương 0,55 điểm phần trăm.

7.2. So sánh với baseline bag-of-tokens (pilot 1.000 mẫu)

Trên cùng reservoir và split WinMET 1.000 mẫu, trung bình hai seed, mô hình attention vượt baseline bag-of-tokens cùng hàm loss về mAP và macro-F1. Top-1/top-3 chưa thắng nhất quán, vì chỉ có 63 mẫu validation có họ đã học.

Phiên bản chuẩn hóa	Mô hình	Validation mAP	Macro-F1
PE-v2	Hybrid Attention	0,408	0,230 (calibrated)
PE-v2	Bag-of-tokens	0,293	0,146 (calibrated)
Process-relations v3	Hybrid Attention	0,357	0,228
Process-relations v3	Bag-of-tokens (cùng loss)	0,254	0,155

7.3. Ablation các thành phần (checkpoint epoch 4)

Ablation thực hiện lúc suy luận trên cùng 181 mẫu validation. Cross-attention giữ nguyên top-1 nhưng đóng góp rõ vào macro-F1, tức giúp các họ hiếm; cạnh đồ thị đóng góp 2,76 điểm top-1.

Cấu hình	Top-1	Top-3	Macro-F1	mAP
Đầy đủ	60,22%	72,93%	24,13%	33,55%
Tắt cross-attention	60,22%	70,72%	14,38%	30,07%
Che cạnh đồ thị, giữ node	57,46%	72,93%	19,88%	29,73%

7.4. Readout và dự đoán có chọn lọc (checkpoint epoch 4)

Ngưỡng chấp nhận được fit trên calibration (216 mẫu) và chỉ đọc trên validation. Khi chỉ trả lời các mẫu đủ tin cậy, độ chính xác vượt 85% nhưng chỉ phủ khoảng một nửa số mẫu; con số này không thay thế top-1 toàn phần.

Cách đọc kết quả	Top-1	Top-3	Top-1 có chọn lọc	Độ phủ
Hybrid, head mô hình	60,22%	72,93%	87,78%	49,7% (90/181)
Hybrid, kNN-5 trên embedding	60,22%	72,93%	90,91%	48,6% (88/181)
Hybrid tắt cross-attention, kNN-5	65,19%	79,56%	91,92%	54,7% (99/181)
PE-only, head mô hình	21,55%	34,81%	không đạt ngưỡng	–
PE-only, kNN-15	56,91%	74,03%	92,19%	35,4% (64/181)

Readout kNN được so sánh trên validation, nên cần cố định trước rồi mới kiểm tra một lần trên test.

7.5. Kết quả trên dữ liệu mở rộng v2 (13/11/2026)

Trên manifest v2 (8.500 mẫu train, 153 họ có ⩾ 5 mẫu), một baseline TF-IDF + logistic regression [14] đạt top-1 71,6% và mAP 0,557, cao hơn mô hình Hybrid Attention sau 6 epoch (64,2% và 0,389). Cả hai được chấm trên cùng 670 mẫu validation có họ trong vocabulary, cùng nhãn, mask và hàm tính metric; tập test chỉ được đọc sau khi đóng băng lựa chọn (mục 7.6).

Mô hình	Feature	Thời gian	Top-1	Top-3	mAP	Macro-F1
Training prior	–	–	17,9%	35,5%	0,008	–

Mô hình	Feature	Thời gian	Top-1	Top-3	mAP	Macro-F1
Hybrid Attention, 6 epoch, d_model 32	Ba nhánh	~50 phút chuẩn bị + ~30 phút/epoch	64,2%	78,1%	0,389	0,288
TF-IDF + LR, C = 16	Chỉ băng chứng tĩnh/signature	15 giây fit	63,6%	79,3%	0,499	0,365
TF-IDF + LR, C = 16	Chỉ event + cặp event liền nhau	163 giây fit	70,7%	85,5%	0,548	0,379
TF-IDF + LR, C = 16 (được chọn)	Tất cả	266 giây fit	71,6%	85,7%	0,557	0,391

Mô hình attention vẫn đang tăng khi dừng: top-1 qua 6 epoch là 50,0 → 55,2 → 58,7 → 61,3 → 59,4 → 64,2%, mAP từ 0,166 lên 0,389. Baseline TF-IDF chạy trọn trên CPU trong 29 phút, gồm 6 phút đọc dữ liệu và 9 cấu hình (script classical_family_baseline.py). Biến thể cosine_balanced đã được bỏ để dành thời gian cho so sánh này.

Ghép điểm hai mô hình (late fusion, w × attention + (1 − w) × TF-IDF, w từ 0 đến 1 bước 0,1) không cải thiện kết quả: w = 0 cho mAP cao nhất, và top-1 giảm dần từ 71,6% (w = 0) xuống 68,1% (w = 0,5) và 64,2% (w = 1). Kiểm tra chéo theo 11 archive block (chọn w trên một nửa, chấm nửa còn lại) cũng chọn w = 0 ở cả hai fold. Cấu hình được đóng băng để chấm test vì vậy là TF-IDF + LR (fusion_v2_selection.json).

Cách đọc kết quả:

- Trên nhãn AVClass hiện tại, phần lớn tín hiệu nằm ở việc có hay không có một API/tham số cụ thể; mô hình tuyến tính học điều này nhanh và tốt. Mô hình attention d_model 32, chỉ 6 epoch, chưa đủ để vượt mức này.
- So sánh "attention thăng bag-of-tokens" ở mục 7.2 vẫn đúng, nhưng baseline ở đó là bag-of-tokens neural train cùng khung; TF-IDF + LR là baseline mạnh hơn và nên là mốc so sánh chính từ nay.
- Chỉ với băng chứng tĩnh, TF-IDF đạt 63,6% top-1. Nhận định "chỉ có tĩnh thì không hơn baseline prior" ở mục 6.3 chỉ đúng cho mô hình attention train có event rồi bị bỏ event lúc suy luận, không đúng cho mô hình train riêng trên feature tĩnh.
- Cấu hình TF-IDF được chọn trong 9 cấu hình theo validation mAP, nên con số 71,6% hơi lạc quan; C = 16 nằm ở biên lưới thử. Kết quả test ở mục 7.6 (70,2–73,2%) cho thấy mức này không bị thổi phồng đáng kể.
- Validation v2 (670 mẫu, 11 block) khác validation v1 (181 mẫu), nên không so trực tiếp 64,2% ở đây với 64,1% của run3.

7.6. Kết quả trên test độc lập (chấm một lần, 13/11/2026)

Trên hai tập test độc lập, TF-IDF + LR đạt 73,2% và 70,2% top-1, khớp với mức 71,6% trên validation; mô hình attention đạt 61,3% và 64,1%. Cấu hình được đóng băng trên validation trước khi đọc test; script fuse_attention_tfidf.py tạo lock file nên không thể chấm lại để chọn mô hình. Cả hai mô hình được báo cáo để minh bạch, không dùng để chọn lại.

Tập test	Mẫu có họ trong vocabulary	Mô hình	Top-1 (Wilson 95%)	Top-3	mAP	Macro-F1
Test v1 (withheld_internal_test, 362 mẫu)	302	TF-IDF + LR (đã chọn)	73,2% (67,9–77,9)	87,4%	0,627	0,393
Test v1	302	Hybrid Attention, 6 epoch	61,3% (55,7–66,6)	79,8%	0,475	0,355
Test v1	302	Training prior	17,2%	–	–	–
Test v2 (withheld_test_v2, 779 mẫu)	668	TF-IDF + LR (đã chọn)	70,2% (66,6–73,6)	86,1%	0,586	0,318
Test v2	668	Hybrid Attention, 6 epoch	64,1% (60,4–67,6)	79,9%	0,390	0,305
Test v2	668	Training prior	20,2%	–	–	–

Lưu ý khi đọc: khoảng tin cậy tính riêng từng mô hình, chưa phải kiểm định cặp; 58–63% mẫu train mới có imphash trùng với test nên con số có thể lạc quan; nhãn vẫn là AVClass. Kết quả lưu ở artifacts/winmet/fusion_v2_test_once.json.

7.7. Phân tích lỗi và các cải tiến không cần train nặng (14/11/2026)

Các biến thể rẻ của TF-IDF + LR trên validation v2 (đổi C, thêm bộ ba event, thêm token bỏ tham số, kNN cosine, ghép theo thứ hạng; 28 biến thể, family_cheap_boosts.py) không nâng top-1: cao nhất 71,9% so với 71,6%, kNN 68,2%, ghép 70,4%. Phân tích lỗi (family_error_analysis.py) cho thấy nút thắt nằm ở dữ liệu và nhãn hơn là mô hình:

Nhóm trên validation (670 report)	Report	Top-1
Họ có ≥ 200 mẫu train	313	89,1%
Họ có 50–199 mẫu	191	58,1%
Họ có 20–49 mẫu	76	55,3%
Họ có 5–19 mẫu	90	53,3%

Nhóm trên validation (670 report)	Report	Top-1
Imphash đã gặp trong train	572	73,8%
Imphash chưa gặp	98	59,2%
TLSH gần nhất ⩽ 30 (kNN theo fuzzy hash)	173	86,1%
TLSH gần nhất ⩽ 50	307	76,9%

Các nhầm lẫn phổ biến nhất đổ về hai họ đông mẫu: snakelogger, taskun, gamarue, noon, msilzilla bị đoán thành agenttesla; injuke, amadey, crifi, deyma thành redline. Nhiều nhãn trong số này là tên crypter hoặc stealer .NET, nên có thể nhãn AVClass đang trộn vỏ đóng gói với payload (suy luận, chưa kiểm chứng). Hướng tiếp cận mới đề xuất: (1) định tuyến lai, dùng nhãn mẫu train gần nhất khi TLSH rất gần và LR cho phần còn lại (ngưỡng cần cố định trước rồi kiểm chứng); (2) xử lý long tail bằng thêm dữ liệu cho họ hiếm, phân tầng nhóm → họ, hoặc học theo prototype; (3) tách vỏ đóng gói và payload bằng thông tin payload đã giải nén của CAPE, sau khi loại mọi tên họ để tránh lộ nhãn.

Hai hướng đầu tiên đã được thử trên validation với kiểm định McNemar ghép cặp theo từng report (tlsh_routed_hybrid.py, long_tail_boosts.py). Định tuyến TLSH (ngưỡng 30, được cả hai nửa cross-fitting chọn) nâng top-1 từ 71,6% lên 72,8% (đúng thêm 18, sai thêm 10, p = 0,18); cân bằng trọng số lớp và chỉnh logit theo tần suất họ không giúp (71,5–71,8%), còn chỉnh logit mạnh ($\tau = 1$) làm tụt xuống 62,1%. Cấu hình chọn bằng cross-fitting đạt 72,7% (p = 0,28 so với LR). Kết luận: với feature và nhãn hiện tại, top-1 họ đã chạm trần khoảng 72–73%; muốn vượt cần thông tin mới (thêm mẫu cho họ hiếm, dữ liệu payload đã giải nén) hoặc xử lý nhãn AVClass trộn vỏ đóng gói và payload, chứ không phải thêm tinh chỉnh mô hình.

Thêm 188 signature hành vi CAPE làm feature cho phân loại họ (dữ liệu v5, signature_feature_family.py) cũng không giúp: 71,8% so với 71,6% (đúng thêm 1 report, p = 1,0). Riêng signature đạt 64,6%, nhưng thông tin trùng với event vì CAPE sinh signature từ chính các event đó. Trên v5, top-1 họ của baseline trùng hẳn v4 (71,64%). Bốn hướng không cần train nặng (tinh chỉnh TF-IDF, kNN và ghép, định tuyến TLSH, xử lý long tail, thêm signature) đều không vượt được khoảng 72–73% một cách có ý nghĩa thống kê, củng cố kết luận rằng trần nằm ở dữ liệu và nhãn.

Kiểm tra nguồn dữ liệu còn lại: hai volume WinMET có 12.738 report, đã dùng 10.640; 2.098 report chưa dùng đều lớn hơn 64 MiB (trung vị 103 MiB) nên đã bị bỏ qua khi chuẩn bị. 1.693 report trong số đó nằm trong block thuộc train, nhưng 1.298 là một họ duy nhất (vbclone); các họ đang bị nhầm nhiều nhất gần như không có thêm mẫu. Vì vậy thêm dữ liệu từ WinMET không giải quyết được các lỗi hiện tại; cần nguồn dữ liệu khác hoặc xử lý lại định nghĩa nhãn (báo cáo riêng với kết quả hiện tại).

Đánh giá song song theo nhóm họ dễ nhầm (family_group_eval.py): nhóm được lập chỉ từ train (dự đoán ngoài fold theo archive block, gộp hai họ khi tỉ lệ nhầm qua lại ≥ 0,30, ngưỡng cố định trước). 153 họ thành 99 nhóm; top-1 theo họ 71,6%, theo nhóm 86,9%. Nhóm thứ hai gồm agenttesla, snakelogger, lokibot, noon, taskun, msilzilla, đúng với phân tích lỗi. Tuy vậy cách gộp bắc cầu tạo một nhóm 27 họ trộn stealer, ransomware, RAT và banker, nên 86,9% bị thổi phồng một phần; con số này chỉ dùng để chỉ ra phần lớn lỗi là nhầm giữa các họ gần nhau, không phải một mức accuracy của hệ thống.

7.8. Quá trình phát triển qua các phiên bản và kết quả cuối cùng

Sau khung ban đầu, công trình đi qua sáu phiên bản. Mỗi phiên bản sửa một vấn đề cụ thể của phiên bản trước và được chấm trên validation của chính nó, nên số liệu giữa các dòng chỉ so được khi cùng dữ liệu.

Phiên bản	Thời gian	Dữ liệu	Thay đổi chính	Kết quả validation
0. Khung ban đầu	04/2026	–	Dashboard XAI, chưa có split và benchmark có kiểm soát	Không dùng làm kết quả
1. Mô hình tĩnh EMBER2024	25–26/10/2026	34.000 mẫu feature tĩnh	Signature–Static Attention (d_model 64), split train/validation/calibration/holdout	Top-1 71,9% so với bag-of-tokens 64,6% và prior 33,1% (520 mẫu); holdout nội bộ 73,2% (1.005 mẫu)
2. WinMET pilot, chuẩn hóa PE-v2	26–27/10/2026	1.000 report, 2 seed	Mô hình ba nhánh + cross-attention; sửa schema, lọc YARA chứa tên họ	mAP 0,408 so với bag 0,293; macro-F1 0,230 so với 0,146
3. Trace process-relations v3	27/10/2026	1.000 report	Nhận diện pseudo-handle, quan hệ cha–con giữa process	mAP 0,357 so với bag 0,254
4. Resource-relations v4, mở rộng	29/10/2026	2.500 report (1.676 train, 181 validation)	Cạnh theo tài nguyên dùng chung; giữ nguyên gán archive block	Epoch 4 (dừng do tắt máy): top-1 60,2%, mAP 0,336

Phiên bản	Thời gian	Dữ liệu	Thay đổi chính	Kết quả validation
5. run3: train đến hội tụ	11/11/2026	Như bản 4	pre_encode và giải phóng cache MPS (14 → 6,5 phút/epoch); readout cố định	Epoch 10: top-1 64,1%, top-3 80,1%; có chọn lọc 90,8% ở độ phủ 54,1%; checkpoint của bản demo
6. Dữ liệu v2 ×5	12–13/11/2026	8.500 train, 670 validation	Nén report trong RAM, giải phóng cache khi validate; thêm baseline TF-IDF + LR	Attention 6 epoch: 64,2%; TF-IDF + LR: 71,6% (mục 7.5)

Kết quả cuối cùng của đề tài:

- Phân loại họ toàn phần trên test độc lập (chấm một lần): TF-IDF + LR đạt 73,2% top-1 trên test v1 và 70,2% trên test v2 (top-3 87,4% và 86,1%); mô hình attention đạt 61,3% và 64,1%. Mục tiêu 85% top-1 chưa đạt.
- Dự đoán có chọn lọc: checkpoint attention run3 đúng 90,8% trên 54,1% số mẫu được chấp nhận và chỉ nhận nhầm 3/47 mẫu thuộc họ ngoài vocabulary.
- Đóng góp của attention: trên cùng khung neural, cross-attention nâng macro-F1 từ 14,4% lên 24,1% và cạnh đồ thị thêm 2,76 điểm top-1 (mục 7.3). Tuy vậy mô hình nhỏ (d_model 32) chưa vượt được baseline tuyến tính mạnh.
- Hệ thống: dashboard chạy cục bộ, không thực thi mẫu, parse PE trong sandbox, tự chọn mô hình theo đầu vào và trả UNKNOWN khi đầu vào ngoài phạm vi đã kiểm chứng.
- Chưa có: nhiều seed cho mô hình trên dữ liệu v2, và đánh giá riêng nhóm mẫu test không trùng imphash với train.

8. Hạn chế và hướng phát triển

Hạn chế lớn nhất hiện nay là khoảng cách giữa top-1 validation (64–72%) và mục tiêu 85%, cùng với việc kết quả vẫn chỉ đo trên validation nhỏ và nhãn yếu.

8.1. Hạn chế

- **Validation nhỏ và tương quan.** Manifest v1 chỉ có 181 mẫu đánh giá được, nằm trong 3 archive block. Chênh lệch dưới 3–4 điểm chủ yếu là nhiễu. Manifest v2 nâng validation lên 783 mẫu trong 11 block.
- **Nhãn yếu.** Nhãn họ lấy từ AVClass consensus trên kết quả antivirus, nên có nhiễu và độ chi tiết không đều. Trần độ chính xác thật chưa được đo.

- **Họ ngoài vocabulary.** Trong 246 mẫu validation v1 có 47 mẫu thuộc họ không đủ 5 mẫu train và 18 mẫu không có nhãn. Hệ thống cần từ chối các mẫu này thay vì gán nhãm.
- **Phụ thuộc nhánh động.** Khi bỏ event lúc suy luận, head của mô hình WinMET không tốt hơn baseline prior. Một mô hình train riêng trên feature tĩnh thì đạt 63,6% top-1 (TF-IDF, mục 7.5), nên file PE upload trực tiếp cần mô hình tĩnh riêng thay vì head của mô hình ba nhánh.
- **Trùng imphash.** Khoảng 58–63% mẫu train mới có imphash xuất hiện trong test, có thể làm kết quả test lạc quan nếu các mẫu này là biến thể gần nhau.
- **Giới hạn phần cứng.** Huấn luyện trên MacBook 16 GB RAM (MPS) giới hạn kích thước mô hình (`d_model` = 32) và số thí nghiệm lặp lại theo seed. Trên dữ liệu v2, mỗi lần chạy mất ~50 phút chuẩn bị và ~30 phút mỗi epoch; máy ngủ khi gập nắp đã làm một lần chạy 6 epoch kéo dài gần 30 giờ.
- **Attention chưa phải giải thích nhân quả.** Trọng số attention cần thêm bằng chứng faithfulness (deletion AOPC, Integrated Gradients [11]) mới được dùng làm lời giải thích.

8.2. Hướng phát triển

Hướng phát triển chia theo ba mức tài nguyên. Mức sau giả định đã làm xong mức trước. Các mốc thời gian là ước lượng kế hoạch, chưa được đo.

Mức	Tài nguyên, thời gian	Việc chính	Kỳ vọng kiểm chứng
A. Hiện tại	MacBook 16 GB (MPS), khoảng 1–2 tuần	Lưu tensor đã encode ra đĩa (bỏ 50 phút chuẩn bị mỗi lần chạy); giảm max_events 2.048 → 1.024 và gom batch theo độ dài; train v2 đến hội tụ (12–15 epoch); train attention đến hội tụ để xem có vượt TF-IDF không (ghép điểm tuyến tính đã thử, không giúp, mục 7.5); bắt đầu học hành vi đa nhãn; cố định readout rồi chấm test cho mô hình đã cải tiến (test hiện tại đã dùng cho lựa chọn ngày 13/11, cần tập test mới hoặc báo cáo rõ là đánh giá lần hai)	Top-1 và mAP trên validation v2 tăng so với mục 7.5; số test đầu tiên kèm khoảng tin cậy Wilson

Mức	Tài nguyên, thời gian	Việc chính	Kỳ vọng kiểm chứng
B. GPU 24–48 GB	Thuê GPU (ví dụ A100/RTX 4090), 2–4 tuần	d_model 128, batch 32; 3–5 seed và báo cáo trung bình ± độ lệch; train trên toàn bộ các volume WinMET; tiền huấn luyện tự giám sát trên trace không nhãn (che event rồi đoán lại); tìm siêu tham số có hệ thống	Mô hình lớn có vượt baseline TF-IDF một cách ổn định qua nhiều seed hay không
C. Nhóm nghiên cứu	Nhiều GPU, 1–3 tháng	Làm sạch nhãn (đối chiếu AVClass2, loss chịu nhiễu, confident learning); chia theo thời gian và nhóm không trùng imphash; bổ sung trace lành tính; kiểm định faithfulness của attention; đánh giá độ bền trước tấn công né tránh (chèn API vô hại vào chuỗi)	Ước lượng được trần độ chính xác do nhiễu nhãn và độ tổng quát thật trên mẫu mới

Mục tiêu 85% top-1 toàn phần chỉ nên đặt lại sau khi đo được trần do nhiễu nhãn (mức C). Trước đó, cách thực tế để đạt độ chính xác cao là dự đoán có chọn lọc: trả lời khi đủ tin cậy và báo UNKNOWN cho phần còn lại (mục 7.4).

8.3. Hướng hành vi: kết quả bước đầu

Riêng phân loại hành vi đa nhãn theo ATT&CK/MBC hiện do luật chuỗi trong evidence.py xác nhận (15 kỹ thuật). Một băng chứng riêng lẻ (một API, import, entropy hay hash) không bao giờ tạo kết quả confirmed: khớp dưới một nửa chuỗi là not_detected, từ một nửa trở lên là suspected, và confirmed cần đủ chuỗi, đúng thứ tự, cùng đích (process khác với pid hợp lệ). Behavior head của mô hình chưa được train vì WinMET thiếu nhãn âm mạnh. Việc cần làm theo thứ tự:

1. Sinh nhãn behavior: chuỗi luật đạt confirmed làm nhãn mạnh, signature/TTP của CAPE làm nhãn yếu; train behavior head đa nhãn và chấm riêng từng kỹ thuật.
2. Chứng minh tác dụng của kết hợp trên bài toán behavior: so mô hình bag (không thứ tự) với mô hình có thứ tự/đồ thị, kèm test chèn từng API riêng lẻ so với cả chuỗi. Thí nghiệm tương tác cặp trên 10 mẫu hiện chưa cho thấy cặp được attention chú ý khác cặp ngẫu nhiên.
3. Đo tỉ lệ nhận nhầm trên trace lành tính, dữ liệu hiện còn thiếu.

4. Mở rộng luật sang thêm kỹ thuật và giữ ánh xạ ATT&CK → MBC → NIST SP 800-53 cho mỗi kỹ thuật mới.

Thống kê nhãn trên 8.500 report train (behavior_label_census.py, 13/11/2026) cho thấy hai trở ngại trước khi train behavior head. Thứ nhất, dữ liệu WinMET đã chuẩn hóa không có nhãn TTP nào từ signature CAPE (0 loại nhãn), nên nguồn nhãn duy nhất là chính các luật; train trên đó chỉ học lại luật và không có cách đo độ chính xác độc lập. Thứ hai, phân bố rất lệch: T1055 chỉ confirmed 3 report, T1547.001 và T1105 không confirmed lần nào (T1105 bị chặn có chủ đích), trong khi các kỹ thuật khám phá hệ thống phổ biến như T1083 (3.899 confirmed) và T1057, T1112 (suspected ở khoảng 80% report) ít giá trị phân biệt. Kiểm tra report gốc trong WinMET volume 1 cho thấy nguồn nhãn độc lập có tồn tại: mỗi report có 6–20 signature CAPE (ví dụ injection_rwx, persistence_autorun, uses_windows_utilities_to_create_scheduled_task) nhưng không kèm trường TTP, nên bước chuẩn hóa hiện bỏ qua chúng. Bước tiếp theo là ánh xạ tên signature sang kỹ thuật ATT&CK theo khai báo của bộ signature cộng đồng CAPE, loại các signature dựa trên phát hiện họ/YARA (cape_detected_threat, procmem_yara, virus) để tránh lộ nhãn, rồi chuẩn bị lại dữ liệu. Report đầu tiên được xem có persistence_autorun trong khi luật T1547.001 chưa bao giờ confirmed. Chẩn đoán trên 300 report có mở khóa autorun: bước mở khóa khớp đúng, nhưng sự kiện ghi giá trị chủ yếu mang đích <REGISTRY_HANDLE_CONFLICT> (1.198 lần) hoặc chỉ còn gốc HKLM/HKCU (507 lần), chỉ 2 lần giữ được cờ AUTORUN. Bộ giải handle trong handles.py đánh dấu xung đột khi đường dẫn ghi khác khóa đã mở, nên mối nối "mở khóa Run → ghi giá trị" bị mất. Đây là lựa chọn thận trọng (không confirmed khi không nối được), nhưng cần sửa quy tắc so khớp giữa khóa cha và giá trị con trong một phiên bản chuẩn hóa mới; đã xác nhận trên report gốc: RegOpenKeyExW mở HKCU\…\CurrentVersion\Run (handle 0x450), rồi RegSetValueExW cùng handle ghi giá trị con …\Run\ajjf trỏ tới một file .exe trong AppData; vì đường dẫn giá trị con khác khóa đã mở và cả hai đều mang cờ autorun, bộ giải handle gán CONFLICT. Cách sửa đề xuất: coi là nhất quán khi đường dẫn ghi bằng đường dẫn khóa đã mở cộng \<tên giá trị>.

Bản chuẩn hóa v5 (cape-resource-relations-v5, 13/11/2026) đã được cài đặt dưới dạng tùy chọn, v4 vẫn là mặc định. v5 nối lệnh ghi giá trị registry với khóa đã mở khi đường dẫn ghi bằng khóa đó cộng một tên giá trị, đồng thời siết lại trường hợp v4 tin handle dù đường dẫn ghi trỏ sang khóa khác (trước đây có thể confirmed nhầm). v5 cũng lưu tên signature CAPE hành vi vào metadata và loại các signature phát hiện họ/YARA/IDS.

Trên 300 report gốc (volume 1)	Confirmed T1547.001	Confirmed và có signature persistence_autorun	Confirmed nhưng không có signature
v4	0	0	0
v5	15	15	0

123/300 report có signature persistence_autorun, nên luật v5 vẫn bảo thủ (độ phủ 15/123) nhưng chưa nhận nhầm so với tham chiếu này. Việc ghép tên signature với kỹ thuật ở đây dựa trên tên gọi, chưa phải bảng ánh xạ chính thức của CAPE. Toàn bộ dữ liệu v2 đang được chuẩn bị lại bằng v5 với đúng các block và mẫu của v4 (chế độ replay).

Trên dữ liệu v5 đầy đủ (8.464/8.500 report train có signature, 218 loại, 52 loại có ≥ 200 mẫu), một baseline hành vi đa nhãn dự đoán 52 signature chỉ từ chuỗi event (TF-IDF + logistic regression, C = 4 cố định, behavior_signature_baseline.py) cho thấy thông tin thứ tự có ích cho bài toán hành vi:

Biểu diễn event (validation, 782 report)	mAP	Micro-F1	Macro-F1
Chỉ đếm API, không thứ tự	0,796	0,854	0,696
Thêm cặp event liền nhau	0,803	0,858	0,702

Thêm thứ tự cải thiện 34/50 signature; bootstrap ghép cặp cho ΔmAP = +0,007 (95%: +0,002 đến +0,013). Mức tăng lớn nhất nằm ở hành vi vốn là chuỗi: antisandbox_sleep (+0,13), infostealer_browser (+0,07). Đây mới là thứ tự cục bộ hai bước; mô hình attention có cửa sổ và đồ thị là bước so sánh tiếp theo. Luật T1547.001 trên validation: 41 confirmed, cả 41 đều có signature persistence_autorun (không nhận nhầm so với tham chiếu này); 188 suspected (184 có signature); bỏ sót 128 report có signature.

Behavior head của mô hình Hybrid Attention (train 6 epoch trên v5, nhãn signature, report winmet_v5_behavior_signatures_e6.json) chưa bắt kịp baseline tuyến tính trên cùng 52 signature và cùng 782 report validation: mAP 0,626, micro-F1 0,703, macro-F1 0,599, so với 0,803 / 0,858 / 0,702; thua ở cả 52 signature. mAP hành vi vẫn tăng qua từng epoch (0,45 → 0,53 → 0,58 → 0,60 → 0,62 → 0,63), nên mô hình chưa hội tụ; d_model 32 cũng phải chia sẻ giữa họ và hành vi. Phân loại họ của cùng mô hình không bị ảnh hưởng (top-1 62,5%, mAP 0,413). Kết luận hiện tại: thông tin thứ tự có ích cho hành vi, nhưng mô hình attention nhỏ chưa khai thác được nó tốt bằng TF-IDF có cặp event; cần train lâu hơn, mô hình lớn hơn (mức B ở bảng trên) hoặc chọn checkpoint theo behavior mAP.

Train tiếp đến 10 epoch và chọn checkpoint theo behavior mAP (winmet_v5_behavior_signatures_e10_bsel.json; epoch 1–6 trùng số với lần chạy trước, xác nhận tái lập được): epoch 10 được chọn với behavior mAP 0,663, micro-F1 0,727, macro-F1 0,619, vẫn thấp hơn baseline 0,803 và chỉ hơn baseline ở 2/52 signature. Đường cong đã chậm lại (0,646 → 0,663 qua 4 epoch cuối), nên train lâu hơn trên máy này khó thu hẹp khoảng cách; cần mô hình lớn hơn hoặc kiến trúc khai thác chuỗi tốt hơn. Cùng mô hình đạt top-1 họ 65,5% và mAP họ 0,464 trên validation v2, cao hơn lần train 6 epoch (64,2%, 0,389) nhưng vẫn dưới TF-IDF (71,6%).

8.4. Dữ liệu có nhãn tin cậy hơn

Phân tích lỗi (mục 7.7) cho thấy trần khoảng 72% nằm ở nhãn và dữ liệu. Điều này phù hợp với MOTIF [15]: trên 3.095 mẫu có nhãn do chuyên gia gán, nhãn AVClass chỉ đúng 46,78% và bỏ phiếu đa số antivirus đúng 62,10%. Hướng tiếp theo vì vậy là bổ sung dữ liệu có nhãn tin cậy hơn và báo cáo song song với WinMET, không thay thế:

Dataset	Dữ liệu	Nhãn	Vai trò trong đề tài
Avast-CTU Public CAPE [16]	48.976 report CAPEv2, không kèm file thực thi	Do Avast gán: 10 họ, 6 loại mã độc, ngày phát hiện	Dataset chính cho hướng họ và hành vi; chia split theo thời gian
MOTIF [15]	3.095 mẫu, 454 họ, feature EMBER v2	Chuyên gia gán, kèm bảng alias	Tập kiểm tra chuẩn cho nhánh tĩnh
MalDICT [17]	Khoảng 5,5 triệu file, feature EMBER	Tag hành vi, platform, khai thác lỗ hổng và packer, suy ra từ nhãn antivirus	Tham khảo cho việc tách nhãn packer và payload

Avast-CTU và MOTIF đã được tải về ngày 14/11/2026 (khóa SHA-256 trong configs/external_datasets.lock.json); chỉ tải report và feature, không tải file nhị phân. Khi chuẩn bị Avast-CTU, các trường có thể chứa tên họ (malfamily_tag, detections, cấu hình CAPE, Suricata) bị loại bỏ, và token chứa tên họ được lọc theo nguyên từ. Ngưỡng kích thước report 512 MiB được chọn để tỉ lệ report bị bỏ ở mỗi họ không vượt quá 1%; ngưỡng 64 MiB từng loại tới 12% mẫu Zeus và 10% mẫu njRAT. Avast-CTU chỉ có 10 họ, nên kết quả trên đó không so trực tiếp với 153 họ của WinMET.

9. Kết luận

Đề tài xây dựng một hệ thống phân loại đa nhãn họ mã độc dựa trên bằng chứng, gồm mô hình Hybrid Attention ba nhánh, luật chuỗi hành vi ánh xạ ATT&CK → MBC → NIST và một dashboard phân tích an toàn. Các kết luận chính:

1. Trên WinMET (153 họ, nhãn AVClass), top-1 trên test độc lập chấm một lần đạt 73,2% / 70,2% với TF-IDF + LR và 61,3% / 64,1% với mô hình attention; mục tiêu 85% chưa đạt.
2. Phân tích lỗi cho thấy trần nằm ở dữ liệu và nhãn: họ có ≥ 200 mẫu train đạt 89,1%, họ ít mẫu chỉ 53–58%, và lỗi dồn vào các họ .NET/crypter dễ nhầm. Năm hướng cải tiến không cần train nặng đều không vượt được khoảng 72–73% một cách có ý nghĩa thống kê.

3. Hệ thống ưu tiên không nhận nhầm: dự đoán có chọn lọc đúng 90,8% ở độ phủ 54,1%; luật T1547.001 (v5) confirmed 41/41 lần khớp signature CAPE.
4. Thứ tự event có ích cho bài toán hành vi (ΔmAP = +0,007, khoảng tin cậy 95% không chứa 0), nhưng mô hình attention nhỏ chưa khai thác tốt bằng TF-IDF có cặp event.
5. Quy trình đánh giá (split theo archive block, chọn mô hình chỉ trên validation, chấm test một lần có khóa, cross-fitting, McNemar) là đóng góp phương pháp quan trọng nhất, giúp các con số trên đáng tin cậy.

Tài liệu tham khảo

1. R. Raducu, A. Villagrasa-Labrador, R. J. Rodríguez, and P. Álvarez, “A dataset of Windows malware execution traces,” *Data in Brief*, 2025, doi: 10.1016/j.dib.2025.112273.
2. R. Raducu, A. Villagrasa-Labrador, R. J. Rodríguez, and P. Álvarez, “WinMET Dataset,” Zenodo, 2025, doi: 10.5281/zenodo.16414116.
3. M. Sebastián, R. Rivera, P. Kotzias, and J. Caballero, “AVclass: A tool for massive malware labeling,” in *Proc. Int. Symp. Research in Attacks, Intrusions, and Defenses (RAID)*, 2016, pp. 230–253.
4. K. O’Reilly *et al.*, “CAPEv2: Malware configuration and payload extraction,” GitHub repository. [Online]. Available: https://github.com/kevoreilly/CAPEv2
5. H. S. Anderson and P. Roth, “EMBER: An open dataset for training static PE malware machine learning models,” arXiv:1804.04637, 2018.
6. R. J. Joyce *et al.*, “EMBER2024 – A benchmark dataset for holistic evaluation of malware classifiers,” in *Proc. ACM SIGKDD Conf. Knowledge Discovery and Data Mining (KDD)*, 2025.
7. A. Vaswani *et al.*, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
8. P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2018.
9. A. N. Angelopoulos and S. Bates, “A gentle introduction to conformal prediction and distribution-free uncertainty quantification,” arXiv:2107.07511, 2021.
10. E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927.
11. M. Sundararajan, A. Taly, and Q. Yan, “Axiomatic attribution for deep networks,” in *Proc. Int. Conf. Machine Learning (ICML)*, 2017.
12. The MITRE Corporation, “MITRE ATT&CK, version 19.2.” [Online]. Available: https://attack.mitre.org
13. Joint Task Force, “Security and privacy controls for information systems and organizations,” NIST SP 800-53 Rev. 5, 2020, doi: 10.6028/NIST.SP.800-53r5.

14. F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.
15. R. J. Joyce, D. Amlani, C. Nicholas, and E. Raff, “MOTIF: A malware reference dataset with ground truth family labels,” Computers & Security, 2022, doi: 10.1016/j.cose.2022.102921.
16. B. Bosansky, D. Kouba, O. Manhal, T. Sick, V. Lisy, J. Kroustek, and P. Somol, “Avast-CTU public CAPE dataset,” arXiv:2209.03188, 2022.
17. R. J. Joyce, E. Raff, C. Nicholas, and J. Holt, “MalDICT: Benchmark datasets on malware behaviors, platforms, exploitation, and packers,” arXiv:2310.11706, 2023.

Tài liệu [1], [2] được đối chiếu trực tiếp với trang Zenodo và kho lưu trữ Universidad de Zaragoza ngày 12/11/2026; các tài liệu còn lại ghi theo thông tin công bố phổ biến và nên kiểm tra lại số trang, tập trước khi nộp.